

Observability with Prometheus & Grafana — Setup Guide

Monitoring the **IT Support Chatbot** on a local Kubernetes cluster, using **Windows / PowerShell**.

The application image is pulled from Docker Hub. You will **not** build anything locally.

Every command in this guide has been run against a working cluster and verified.

Concepts

The Four Pillars

Observability rests on four pillars:

Pillar	Purpose
Monitoring	Track metrics — CPU, memory, network of deployed apps
Logging	Collect error/event logs to see what's happening
Tracing	Follow a request's path to find where an error originated
Alerting	Get notified when e.g. CPU crosses 90%

This guide covers **Monitoring** and **Alerting**.

For **Logging**, use `kubectl logs`. For **Tracing**, this app has LangSmith hooks in `config.py`.

Tool Roles

Prometheus — a time-series database. It scrapes metrics (CPU% at t=5min, t=10min...) and stores them against time. Two parts: a scraper pulling data from the cluster, and a query engine (**PromQL**).

Grafana — visualization. Turns Prometheus data into dashboards.

Node Exporter — runs on each node, exports that node's CPU/memory/network/IO to Prometheus.

kube-state-metrics — exports Kubernetes *object* state: replica counts, pod restarts, pending pods. Node Exporter reports the *machine*; kube-state-metrics reports the *cluster's intent*.

Helm — the Kubernetes package manager. A one-line definition worth memorising: "*Helm is a package manager for Kubernetes manifest files, where you can install, manage, delete, and uninstall repositories and any application to be deployed on Kubernetes.*" Packaged manifests are called **Helm charts**.

kind — *Kubernetes IN Docker*. Runs each cluster node as a Docker container on your laptop.

The key idea: Prometheus *pulls*. Your application never pushes metrics to Prometheus; Prometheus reaches out and scrapes them on a schedule.

How This Setup Is Arranged

Everything runs **locally on Windows + Docker Desktop**. There are no cloud servers to provision and no firewall rules to open — **every service is reached at localhost**.

Two conventions used throughout, worth knowing before you start:

- The application runs in the **it-support** namespace, not **default**. Every PromQL query in this guide therefore filters on **namespace="it-support"**.
- The monitoring stack is installed from a **values file** rather than a long chain of **--set** flags, so the configuration is reviewable and repeatable.

Step 1 — Prerequisites

Docker Must Be Running

Docker Desktop must be installed and running before anything else. kind builds your cluster nodes as Docker containers, so nothing here works without it.

On Windows there is nothing to configure beyond having Docker Desktop running.

```
docker ps
```

A table of column headers — even with no rows — means Docker is up. If you see **Cannot connect to the Docker daemon**, open Docker Desktop and wait for the whale icon to stop animating.

Set Docker Desktop Memory to 8 GB

Settings → Resources → Memory → 8 GB → Apply & Restart.

Below about 6 GB you get etcd timeouts and pods stuck in **Pending**. That is memory pressure, not a broken install.

Verify the Toolchain

```
docker ps
```

```
kind version
```

```
kubectl version --client
```

```
helm version
```

Expected:

```
kind v0.32.0 go1.26.3 windows/amd64
Client Version: v1.31.0
version.BuildInfo{Version:"v3.13.1", ...}
```

Step 2 — Install the Tools

Skip this step if the four commands above all printed versions.

Windows

Windows 10 and 11 include **winget**. Run these in **PowerShell**:

```
winget install -e --id Kubernetes.kind
```

```
winget install -e --id Kubernetes.kubectl
```

```
winget install -e --id Helm.Helm
```

Close and reopen PowerShell afterwards so the updated PATH takes effect.

Using Chocolatey instead:

```
choco install kind kubernetes-cli kubernetes-helm -y
```

macOS

```
brew install kind kubectl helm
```

Linux

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1"
sudo install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1
```

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash
```

Step 3 — Create the kind Cluster

The cluster layout is defined by a config file in this repository:

```
cd d:\Ness-AIBootcamp\ness_github_actions
```

```
kind create cluster --name ness-obs --config k8s/kind/kind-cluster.yaml
```

```
kubect1 get nodes
```

kind creates Docker containers as Kubernetes nodes — **1 control-plane + 2 workers**. Wait for all three to show **Ready**:

NAME	STATUS	ROLES	AGE	VERSION
ness-obs-control-plane	Ready	control-plane	21m	v1.36.1
ness-obs-worker	Ready	<none>	21m	v1.36.1
ness-obs-worker2	Ready	<none>	21m	v1.36.1

Confirm they exist as containers:

```
docker ps
```

Port Mappings

Two services cannot share the same NodePort. The cluster config assigns each one a distinct port and maps it straight to your host:

In-cluster NodePort	Host URL
30000	http://localhost:9090 — Prometheus
31000	http://localhost:3000 — Grafana
32000	http://localhost:9093 — Alertmanager
31446	http://localhost:8501 — Chatbot

Because of these mappings **you do not need `kubectl port-forward` at all**. If you ever work on a cluster without them, the Port Reference near the end of this guide has the port-forward commands.

Port mappings are fixed at cluster-creation time. Changing them means recreating the cluster.

Step 4 — Add Helm Repos

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo add stable https://charts.helm.sh/stable
```

```
helm repo list
```

```
helm repo update
```

Step 5 — Install the kube-prometheus-stack

One chart installs all five components: Prometheus, Grafana, Alertmanager, Node Exporter, and kube-state-metrics.

First make sure you are in the project directory. The `-f` path below is relative, so running this from anywhere else fails with `open k8s/kind/monitoring-values.yaml: The system cannot find the path specified`:

```
cd d:\Ness-AIBootcamp\ness_github_actions
```

Note that a bare `cd` with no argument moves you to your **home** directory in PowerShell — it does not return you to the project. Always pass the full path.

Confirm the values file is visible from where you are standing:

```
ls k8s\kind\monitoring-values.yaml
```

Then install:

```
helm install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace -f k8s/kind/monitoring-values.yaml --timeout 15m
```

If you see **cannot re-use a name that is still in use**, the release is already installed. Either skip ahead to Step 6, or remove it first with `helm uninstall monitoring -n monitoring` followed by `kubectl delete namespace monitoring`, wait until `kubectl get ns monitoring` reports `NotFound`, then re-run the install.

Reading that command:

- `monitoring` — the name of this installation (a Helm *release*)
- `--create-namespace` — creates the `monitoring` namespace, so no separate `kubectl create namespace` is needed
- `-f k8s/kind/monitoring-values.yaml` — all settings, instead of a long `--set` chain
- `--timeout 15m` — several container images must download

Why a values file instead of `--set` flags? The same settings can be passed as a long chain of `--set` arguments on one line, but a values file can be reviewed, diffed, and re-applied with `helm upgrade`. It also sets memory limits suited to a laptop and disables four components that cannot be scraped on `kind` — see Step 7.

Success:

```
NAME: monitoring
STATUS: deployed
REVISION: 1
```

Verify

```
kubectl wait --for=condition=Ready pod --all -n monitoring --timeout=300s
```

```
kubectl get pods -n monitoring
```

You should see **8 pods** — Alertmanager, Prometheus, Grafana, node-exporter, kube-state-metrics, and the operator:

NAME	READY	STATUS
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running
monitoring-grafana-...	3/3	Running
monitoring-kube-prometheus-operator-...	1/1	Running
monitoring-kube-state-metrics-...	1/1	Running
monitoring-prometheus-node-exporter-...	1/1	Running
monitoring-prometheus-node-exporter-...	1/1	Running
monitoring-prometheus-node-exporter-...	1/1	Running
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running

node-exporter appears three times — once per node. That is a DaemonSet: exactly one copy on every node.

Step 6 — Deploy the Application

Now deploy the application that will be monitored — the **IT Support Chatbot**, pulled from Docker Hub as image `vijaynvb/it-support-chatbot:latest`.

Create the Namespace

```
kubectl create namespace it-support
```

Create the Credentials Secret

The chatbot reads AWS Bedrock credentials from environment variables:

```
kubectl create secret generic aws-credentials -n it-support --from-literal=AWS_ACCESS_KEY_ID=your-key-here --from-literal=AWS_SECRET_ACCESS_KEY=your-secret-here --from-literal=AWS_DEFAULT_REGION=us-east-1
```

Placeholder values are fine for this guide. The web page loads and every metric works; only *sending a chat message* needs real Bedrock keys. Replace them if you want the chat feature to answer.

Apply the Manifests

```
kubectl apply -n it-support -f k8s/kind/chatbot.yaml
```

```
kubectl rollout status deployment/it-support-chatbot -n it-support --timeout=5m
```

The image is about 520 MB, so the first pull takes a few minutes.

```
kubectl get all -n it-support
```

Wait until the pod is **Running**:

NAME	READY	STATUS	RESTARTS	AGE
pod/it-support-chatbot-79fb897bfb-j5l dp	1/1	Running	0	2m

Open **http://localhost:8501** — the chatbot interface loads.

Step 7 — Explore Prometheus

Open **http://localhost:9090**

Check That Scraping Works

Status → Target health. Every endpoint should show **UP** with its **/metrics** path:

```
up 1 apiserver          up 9 kubelet
up 2 coredns            up 3 node-exporter
up 1 kube-state-metrics up 2 prometheus
```

Why no DOWN targets? On kind, **kube-controller-manager**, **kube-scheduler**, **kube-proxy**, and **etcd** bind to addresses unreachable from the pod network. Left enabled they show as permanently DOWN — an alarming red wall that means nothing. The values file disables them, so a DOWN target here is a *real* problem. On EKS you would leave them enabled.

Run PromQL Queries

Go to **Query**, paste an expression, press **Execute**.

Start with the simplest metric — returns **1** for every healthy target:

```
up
```

CPU as a percentage of total cluster cores:

```
sum(rate(container_cpu_usage_seconds_total{namespace="it-support"}[1m])) /  
sum(machine_cpu_cores) * 100
```

CPU in cores, per pod:

```
sum(rate(container_cpu_usage_seconds_total{namespace="it-support",container!=""}  
[2m])) by (pod)
```

Memory per pod:

```
sum(container_memory_working_set_bytes{namespace="it-support",container!=""}) by  
(pod)
```

Network in and out, per pod:

```
sum(rate(container_network_receive_bytes_total{namespace="it-support"}[5m])) by  
(pod)
```

```
sum(rate(container_network_transmit_bytes_total{namespace="it-support"}[5m])) by  
(pod)
```

Node CPU busy percentage:

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) by (instance) * 100)
```

Restart counts — the fastest signal that something is crash-looping:

```
sum(kube_pod_container_status_restarts_total{namespace="it-support"}) by (pod)
```

Always check the namespace filter. Many published PromQL examples use `namespace="default"`, which returns nothing here — this chatbot runs in `it-support`. Every query above already uses the right one.

Switch to the **Graph** tab and set the range to **15 minutes** rather than 1 hour — the default flattens a young cluster into an empty line.

Step 8 — Generate Load to Watch Metrics Move

Metrics are far more convincing when you make them move. Send continuous traffic to the app and watch network and CPU spike in real time.

```
kubectl run loadgen -n it-support --image=busybox:1.36 --restart=Never --command -
- sh -c "while true; do wget -q -O /dev/null http://it-support-chatbot/; done"
```

Wait **60 seconds** — one scrape interval must pass before Prometheus sees anything.

Re-run the network query in Prometheus:

```
sum(rate(container_network_receive_bytes_total{namespace="it-support"}[2m])) by
(pod)
```

Switch to **Graph** and the line jumps. Measured on this setup, the chatbot went from **0.01 to 0.407 CPU cores** and **129.7 KB/s** network under load.

You can also open **http://localhost:8501** and click around the app to add real traffic.

Clean up when done:

```
kubectl delete pod loadgen -n it-support
```

Git Bash users: run this from **PowerShell**. Git Bash rewrites `sh -c` into a Windows path and the pod dies with `stat C:/Program Files/Git/usr/bin/sh: no such file or directory`. If you must use Git Bash, prefix the command with `MSYS_NO_PATHCONV=1`.

Step 9 — Log In to Grafana

Open **http://localhost:3000**

- Username: `admin`
- Password: `prom-operator`

To retrieve it from the secret instead:

```
$p = kubectl get secret -n monitoring monitoring-grafana -o jsonpath="
{.data.admin-password}"
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($p))
```

On macOS or Linux:

```
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-  
password}" | base64 -d
```

Step 10 — Grafana Concepts

Grafana has only two concepts you need at the start: **a data source supplies data, a dashboard displays it.** That is the whole model.

How the Data Sources Got There

You do not add the data sources yourself. Both Prometheus and Alertmanager were configured automatically by the `helm install` in Step 5, along with all 27 dashboards. This is called **provisioning**, and it works like this:

1. The Helm chart creates a **ConfigMap** holding the data source definitions, tagged with the label `grafana_datasource=1`
2. The Grafana pod runs a **sidecar container** that continuously watches the cluster for ConfigMaps carrying that label
3. When it finds one, it copies the contents into Grafana's provisioning folder
4. Grafana reads that folder on startup and creates the data sources

This is why the Grafana pod shows **3/3** containers rather than 1/1 — one sidecar for data sources, one for dashboards, plus Grafana itself:

Container	Watches for label	Writes into
<code>grafana-sc-datasources</code>	<code>grafana_datasource</code>	<code>/etc/grafana/provisioning/datasources</code>
<code>grafana-sc-dashboard</code>	<code>grafana_dashboard</code>	<code>/tmp/dashboards</code>
<code>grafana</code>	—	the Grafana server itself

See it for yourself — the ConfigMap the chart created:

```
kubectl get configmap -n monitoring -l grafana_datasource=1
```

And its actual contents, which is exactly what Grafana loaded:

```
kubectl get configmap -n monitoring -l grafana_datasource=1 -o  
jsonpath='{.items[*].data}'
```

Two consequences worth remembering:

- **Never click "Add new data source"** for Prometheus or Alertmanager. They already exist, and adding another creates a broken duplicate.
- **Provisioned data sources are self-healing.** Delete one in the UI and it reappears when Grafana restarts, because the ConfigMap remains the source of truth. To change one for real, edit the Helm values and run `helm upgrade`.

Check the Data Sources

Open the menu (≡) → **Connections** → **Data sources**.

You should see exactly two entries, **Prometheus** and **Alertmanager**. There is nothing to add — but it is worth knowing what they contain, because this is the connection everything else depends on.

What Prometheus Is Configured With

Click **Prometheus**. These are the values in use:

Field	Value	Meaning
Name	Prometheus	What you pick from the dropdown when building a panel
Default	true	New panels select it automatically
URL	http://monitoring-kube-prometheus-prometheus.monitoring:9090	Where Grafana sends queries
Access	proxy (Server)	Grafana's backend calls Prometheus, not your browser
HTTP method	POST	Allows long queries that would overflow a URL
Scrape interval	30s	Tells Grafana how far apart data points are

Scroll to the bottom and click **Save & test**. It reports:

Successfully queried the Prometheus API.

Understanding the URL is the key to fixing most connection problems. `monitoring-kube-prometheus-prometheus` is the Kubernetes Service name, `.monitoring` is its *namespace*, and `9090` is the *service port*. Grafana runs inside the cluster, so it uses this internal address — **not** `localhost:9090`, which only works from your browser.

What Alertmanager Is Configured With

Click the back arrow, then **Alertmanager**:

Field	Value	Meaning
Name	Alertmanager	Identifier used by alerting dashboards
URL	http://monitoring-kube-prometheus-alertmanager.monitoring:9093	Internal address, same naming pattern as above
Implementation	Prometheus	Tells Grafana this is upstream Alertmanager, not Grafana's built-in alerting
Receive Grafana alerts	off	Grafana does not forward its own alerts here

Do not click "Save & test" on Alertmanager. Unlike Prometheus it has no health-check endpoint, so the button reports an error even when the data source is working perfectly. Verify it instead by opening a dashboard that uses it — see below.

Verify Both Are Actually Working

Configuration screens can look right while the connection is broken. Two checks that prove it end to end:

For Prometheus — open ☰ → **Explore**, select **Prometheus** in the top-left dropdown, enter **up** in the query box, and press **Shift + Enter**. A table of results means Grafana can query Prometheus.

For Alertmanager — open ☰ → **Dashboards** → **Alertmanager / Overview**. If panels show data, the connection works.

If a Data Source Is Missing or Broken

If you experimented with **Add new data source** you may have left a half-configured entry behind, often named **alertmanager-1** with an empty URL. It will fail with:

```
Invalid data source URL: ""
```

Delete any such entry: open it, scroll to the bottom, click **Delete**. Keep only the two the chart created.

To confirm what the chart provisioned, read it straight from the cluster:

```
kubectl get configmap -n monitoring -l grafana_datasource=1 -o
jsonpath='{.items[*].data}'
```

That prints the provisioning file containing both data source definitions with their URLs.

Optional — User Management

Administration → **Users and access** → **Users** → **New user**.

Set name, email, username, password. Then **Change Role** → **Viewer / Editor / Admin** to give read-only access to others.

Step 11 — Build a Dashboard Manually

Create the Dashboard

1. Click the menu icon (≡) top-left → **Dashboards**
2. Click **New** (top-right) → **New dashboard**
3. Click + **Add visualization**
4. Select the **Prometheus** data source

Panel 1 — CPU Usage

5. In the query editor below the graph, click **Code** on the right to type PromQL directly
6. Paste this query:

```
sum(rate(container_cpu_usage_seconds_total{namespace="it-support",container!=""}  
[2m])) by (pod)
```

7. Press **Shift + Enter** to run it — a line appears
8. On the right panel, set **Title** to **Chatbot CPU Usage**
9. Set **Legend** to **Custom** with value `{{pod}}` so each line is named by its pod
10. Set the time range (top-right) to **Last 15 minutes**
11. Click **Save dashboard**, name it **IT Support Chatbot**, click **Save**

No unit is needed on this panel. The query already returns CPU cores, and Grafana has no built-in "cores" unit — typing it only offers "*Custom unit: cores*", which appends the word as a label rather than formatting anything. The official Kubernetes dashboards leave the unit unset on their CPU panels too.

The **metrics browser** is the alternative to Code mode: it lets you pick a metric such as `container_cpu_usage_seconds_total` from a list and filter by namespace via dropdowns. Use whichever you prefer.

Panel 2 — Memory Usage

12. Click **Edit** → **Add** → **Visualization**
13. Switch to **Code** mode and paste:

```
sum(container_memory_working_set_bytes{namespace="it-support",container!=""}) by
```

(pod)

14. Press **Shift + Enter**

15. Set **Title** to **Chatbot Memory Usage**

16. Under **Standard options** → **Unit**, type **bytes** and select **bytes(IEC)** from the *Data* group — the numbers become KB/MB/GB instead of raw bytes

17. Click **Back to dashboard** → **Save dashboard**

Picking units: the Unit box is a searchable list, and anything it offers as "*Custom unit: ...*" is **not** a real unit — it just appends that text to the number. Only select entries that appear under a group heading such as *Data* or *Misc*.

Panel 3 — A Single Number

Graphs show trends; a big number answers "is it healthy right now".

18. Click **Add** → **Visualization**

19. Switch to **Code** mode and paste:

```
sum(kube_pod_status_ready{namespace="it-support",condition="true"})
```

20. Press **Shift + Enter**

21. At the **top-right** there is a visualization dropdown showing **Time series**. Change it to **Stat**

22. Set **Title** to **Pods Ready**

23. Click **Back to dashboard** → **Save dashboard**

Drag panel edges to resize and title bars to rearrange. You now have a three-panel dashboard you built yourself.

Step 12 — Import a Prebuilt Dashboard

This is how the polished dashboards are produced.

1. Go to **<https://grafana.com/grafana/dashboards/>**
2. Search for a Kubernetes dashboard you like
3. Copy its **dashboard ID** (a number)
4. In Grafana: **Dashboards** → **New** → **Import**
5. Paste the ID → **Load**

6. Select **Prometheus** as the data source → **Import**

Popular IDs:

ID	Dashboard
315	Kubernetes cluster monitoring
1860	Node Exporter Full
12740	Kubernetes Monitoring
13332	kube-state-metrics v2

Already Installed

This chart ships **27 dashboards preloaded** — check **Dashboards** before importing. The most useful:

Dashboard	Shows
Node Exporter Full	Everything about a node — CPU, RAM, disk, network
Kubernetes / Compute Resources / Namespace (Pods)	Set namespace to it-support to isolate the chatbot
Kubernetes / Compute Resources / Cluster	Whole-cluster utilisation
Kubernetes / Networking / Namespace (Pods)	Per-pod traffic
Alertmanager / Overview	Alert volume and silences

To view the chatbot specifically: open **Kubernetes / Compute Resources / Namespace (Pods)**, set the **namespace** dropdown to **it-support**, and set the range to **Last 15 minutes**.

Step 13 — Alerting

Open **<http://localhost:9090/alerts>**

134 alert rules ship preloaded. No configuration needed.

The Watchdog Alert

One alert fires permanently: **Watchdog**. This is intentional.

Watchdog is a heartbeat — it always fires, so if your notification pipeline works you should *always* receive it. Silence from Watchdog means the alerting path itself is broken.

Alerts Relevant to This Application

Alert	Fires When
KubePodCrashLooping	Pod restarting repeatedly — often bad AWS credentials

Alert	Fires When
<code>KubePodNotReady</code>	Pod stuck non-Ready over 15 minutes
<code>KubeDeploymentReplicasMismatch</code>	Desired $\neq$ available replicas
<code>KubeContainerWaiting</code>	Stuck pulling an image — wrong tag or private repo
<code>CPUThrottlingHigh</code>	Container throttled against its CPU limit

Open **`http://localhost:9093`** for Alertmanager, which receives firing alerts and decides who to notify. By default it only displays them. Routing to Slack or email requires editing its configuration.

PowerShell Quick Reference

Linux / macOS	Your PowerShell equivalent
<code>cmd &</code>	<code>Start-Job { cmd }</code>
<code>curl -Lo file url</code>	<code>curl.exe -Lo file url</code>
<code>base64 -d</code>	<code>[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String(\$x))</code>
<code>chmod +x</code>	not needed
Cloud firewall rules	not needed — use <code>localhost</code>
<code>get-helm-3</code> script	<code>winget install Helm.Helm</code>

Managing forwards:

`Get-Job`

`Stop-Job *`

`Remove-Job *`

Jobs die when the PowerShell window closes, so re-run the `Start-Job` lines each session. The kind port mappings from Step 3 survive, which is why they are the primary access path in this guide.

Port Reference

Service	Port-forward	URL	Credentials
Prometheus	<code>9090:9090</code>	<code>http://localhost:9090</code>	—

Service	Port-forward	URL	Credentials
Grafana	3000:80	http://localhost:3000	admin / prom-operator
Alertmanager	9093:9093	http://localhost:9093	—
Chatbot	8501:80	http://localhost:8501	—

The rule that trips people up: in `kubectl get svc` output, `9090:30000/TCP` reads **servicePort:nodePort**. Port-forward targets the **left** number.

Using Port-Forward

The port mappings above mean you never need `kubectl port-forward` in this guide. You will need it on any cluster set up without them, so the commands are here for reference.

Linux & becomes `Start-Job` on PowerShell:

```
Start-Job { kubectl port-forward svc/monitoring-kube-prometheus-prometheus -n monitoring 9090:9090 --address=0.0.0.0 }
```

```
Start-Job { kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80 --address=0.0.0.0 }
```

Confirm they bound, looking for `Forwarding from 0.0.0.0:9090 -> 9090`:

```
Get-Job
```

```
Receive-Job -Id 1 -Keep
```

Targeting the nodePort instead of the servicePort fails with `Service does not have a service port 30000`. The correct values for this install:

Service	Ports shown	Port-forward uses
monitoring-kube-prometheus-prometheus	9090:30000	9090
monitoring-grafana	80:31000	80
monitoring-kube-prometheus-alertmanager	9093:32000	9093

Service names come from the Helm release name. This guide installs the release as `monitoring`, so every service is prefixed `monitoring-*`. Run `kubectl get svc -n monitoring` to see the actual names if you used a different release name.

Troubleshooting

Symptom	Cause and fix
<code>docker ps</code> fails	Docker Desktop not running. Open it, wait for the whale icon to settle.
<code>kind: not recognized</code>	Close and reopen the terminal so PATH updates.
<code>open k8s/kind/monitoring-values.yaml</code> : The system cannot find the path specified	You are not in the project directory. Run <code>cd d:\Ness-AIBootcamp\ness_github_actions</code> first. A bare <code>cd</code> goes to your home directory, not the project.
cannot re-use a name that is still in use	The Helm release already exists. Skip the install, or <code>helm uninstall monitoring -n monitoring</code> and delete the namespace first.
does not have a service port 30000	30000 is the nodePort. Use the servicePort — see the Port Reference.
Pods stuck <code>Pending</code> , etcd timeouts	Docker Desktop memory pressure. Raise to 8 GB.
<code>ImagePullBackOff</code>	Cannot download the image. Check internet, then <code>kubectl describe pod -n it-support</code> .
<code>CrashLoopBackOff</code>	Check <code>kubectl logs -n it-support deploy/it-support-chatbot</code> .
Targets DOWN for controller-manager / scheduler / etcd	Expected on kind — disabled in the values file. Re-enable only on EKS.
Grafana panels empty	Range too wide for a young cluster. Set Last 15 minutes .
PromQL returns nothing	Check the namespace filter — this app is in <code>it-support</code> , not <code>default</code> .
<code>stat C:/Program Files/Git/usr/bin/sh</code>	Git Bash path mangling. Use PowerShell or set <code>MSYS_NO_PATHCONV=1</code> .
Chat replies fail, page loads	Expected with placeholder AWS credentials. Monitoring is unaffected.

Diagnostic commands:

```
kubectl get all -n monitoring
```

```
kubectl logs -n it-support deploy/it-support-chatbot --tail=50
```

```
kubectl describe pod -n it-support
```

Cleaning Up

Remove only the monitoring stack:

```
helm uninstall monitoring -n monitoring
```

```
kubectl delete namespace monitoring
```

Remove everything, freeing all memory:

```
kind delete cluster --name ness-obs
```

Nothing in this guide creates cloud resources, so there are no leftover charges — unlike the EKS pipeline documented in [docs/PIPELINE_SETUP.md](#), which bills for as long as it runs.

Command Reference

Task	Command
Create cluster	<code>kind create cluster --name ness-obs --config k8s/kind/kind-cluster.yaml</code>
List nodes	<code>kubectl get nodes</code>
Add Helm repo	<code>helm repo add prometheus-community https://prometheus-community.github.io/helm-charts</code>
Install stack	<code>helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace -f k8s/kind/monitoring-values.yaml</code>
Check monitoring pods	<code>kubectl get pods -n monitoring</code>
Deploy app	<code>kubectl apply -n it-support -f k8s/kind/chatbot.yaml</code>
Check app	<code>kubectl get all -n it-support</code>
View logs	<code>kubectl logs -n it-support deploy/it-support-chatbot</code>

Task	Command
Grafana password	<code>kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}"</code>
Uninstall stack	<code>helm uninstall monitoring -n monitoring</code>
Delete cluster	<code>kind delete cluster --name ness-obs</code>